

Advances in telephony-based speech recognition

M Pawlewski, B P Milner, S A Hovell, D G Ollason, S P A Ringland, K J Power, S N Downey and J Bridges

There are many factors which affect the performance of automatic speech and speaker recognition systems. On its journey from the human to the machine the speech signal encounters several sources of corruption. These include articulation effects such as stress, environmental effects such as noise, and channel effects inherent in the telephony network. The recognition task itself introduces another dimension of difficulty. Although isolated-word recognition is relatively straightforward, natural language input remains a formidable challenge. Five areas of research have been selected which illustrate the diverse field and demonstrate the key issues which must be addressed in order for the technology to flourish.

1. Introduction

In recent years, people's expectations of speech and speaker recognition systems have grown rapidly [1]. As recognition tasks become more and more demanding the underlying core technologies have had to evolve accordingly. In the early 1980s speech recognisers were only expected to cope with limited, isolated-word vocabularies. However, as components of natural language systems, modern recognisers are now required to deal with continuously spoken sentences, while operating in adverse conditions, for example people telephoning from moving vehicles. As a consequence, today's speech recognisers must incorporate many new techniques, together with improvements over their predecessors.

At its simplest level, a speech recogniser consists of just two components, the front end feature extractor and the

pattern classifier, or parser. These fundamental components are described in Power [2]. However, for the recogniser to operate in difficult conditions additional components are required. Figure 1 illustrates the typical constituents of a robust recognition system.

To compensate for environmental distortions, such as noise and non-ideal channels, the speech signal can be enhanced after the parameterisation stage, or alternatively the speech models themselves can be modified to reflect the prevailing conditions using model adaptation techniques. To provide robustness to unexpected words, adjustments may be made to the modelling components and additional post-processing stages included after parsing.

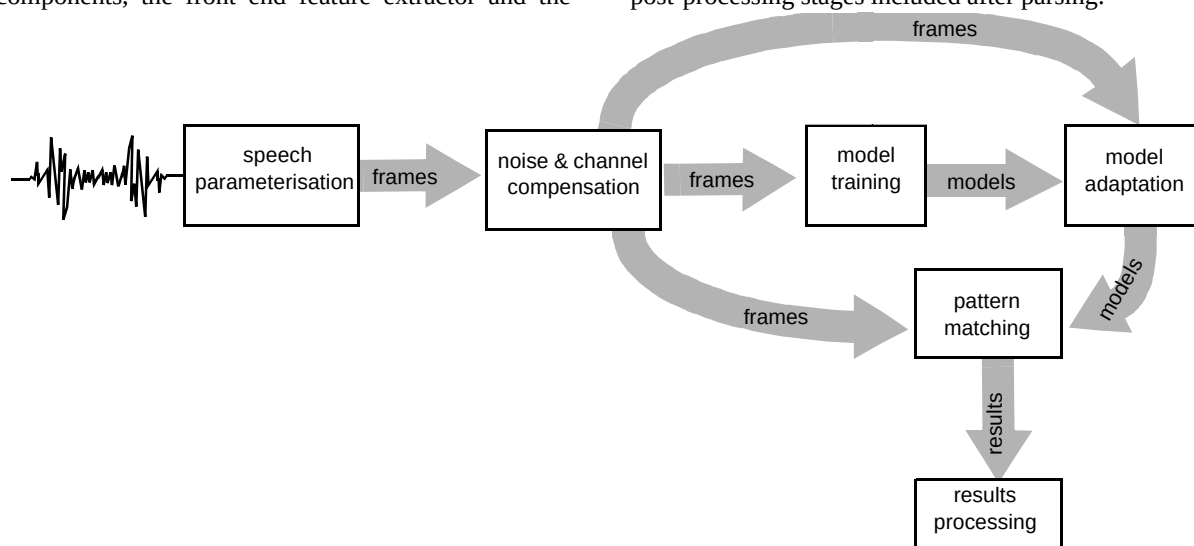
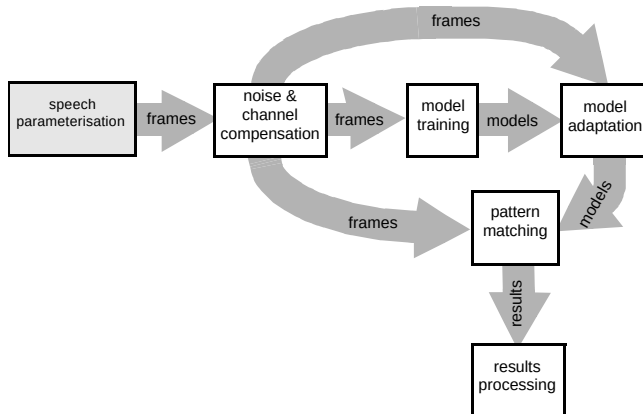


Fig 1 Constituents of a robust speech recognition system.

The following sections of this paper present some of the technical advances which differentiate today's recognisers from those of only a few years ago. The individual sections are written so that they may be read either independently or as a logical sequence.

2. Advances in feature extraction



The first stage of an automatic speech recognition system is parameterisation, or feature extraction. This is the process whereby the time domain speech signal is transformed into a representation containing the key information required for classification.

In the time-domain, speech is highly correlated and contains much redundant information, e.g. that of the identity of the talker, which is not required in speaker-independent speech recognition. It is important to select the feature extraction process carefully, in order to obtain the most useful type of speech feature. Parsons [3] lists six criteria which good features for pattern recognition should possess:

- vary widely from class to class,
- insensitive to extraneous variables,
- stable over long periods of time,
- frequently occurring,
- easy to measure,
- not correlated with other features.

It is difficult to find features which meet all of these requirements, and a compromise is usually made.

2.1 Design of a high performance front end

After feature extraction, successive feature vectors are still correlated. However, a well-known deficiency of hidden Markov models (HMMs) is the lack of an efficient mechanism for the utilisation of this correlation. The left-right HMM provides a temporal structure for modelling the

time evolution of speech spectral characteristics from one state into the next, but within each state the observation vectors are assumed to be independent and identically distributed (IID). The IID assumption states that there is no correlation between successive speech vectors. This implies that within each state the speech vectors are associated with identical probability density functions (pdf) which have the same mean and covariance. This further implies that the spectral-time trajectory within each state is a randomly fluctuating curve with a stationary mean. However, in reality the spectral-time trajectory clearly has a definite direction as it moves from one speech event to the next. This violation of the IID assumption contributes to a limitation in the performance of HMMs. Including some temporal information into the speech feature contradicts the assumption that speech is a stationary-independent process, and can be used to improve recognition performance.

The conventional method of including temporal information is to augment the standard speech feature with first and second order time derivatives [4]. An alternative method for including temporal information is to use a matrix-based feature, namely the cepstral-time matrix [5], which implicitly includes speech dynamics. Several other features exist which can offer increased performance. These work by incorporating extra information into the speech feature, such as dynamic and cross-spectral information. This section describes several of these 'high performance' features.

2.2 MFCCs and differential parameters

Currently the most popular feature set used in speech recognition is the Mel frequency cepstral coefficient (MFCC), augmented with higher-order time derivatives. Typically the feature vector comprises eight MFCCs plus eight first-derivative MFCCs and eight second-derivative MFCCs. The first derivative is computed from the instantaneous cepstrum using regression:

$$\partial c_t(n) = \frac{\sum_{k=-K}^K k c_{t+k}(n)}{\sum_{k=-K}^K k^2} \quad \text{..... (1)}$$

where $\partial c_t(n)$ is the first derivative of the n th MFCC at time frame t , and $c_{t+k}(n)$ is the n th coefficient of the $t+k$ th cepstral vector.

In a similar manner, the second derivative, $\partial^2 c_t(n)$, is calculated using a weighted summation of the first derivative, typically where the window width $K=1$:

$$\partial^2 c_t(n) = \partial c_{t+1}(n) - \partial c_{t-1}(n) \quad \text{..... (2)}$$

The first derivative is often referred to as the velocity of the cepstra through time and the second derivative as the acceleration. Figure 2 illustrates a typical speech vector, comprising instantaneous, velocity and acceleration cepstra.

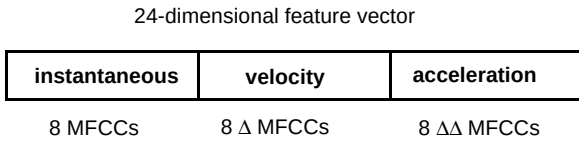


Fig 2 Cepstral feature vector.

2.3 Cepstral-time matrices

An alternative method for including the temporal information of a speech signal is to use a cepstral-time matrix. A cepstral-time matrix, $c_t(m,n)$, is obtained either by applying a 2-D discrete cosine transform (DCT) to a spectral-time matrix or by applying a 1-D DCT to a grouping of conventional MFCC speech vectors [5]. M log filter bank vectors are stacked together to form a spectral-time matrix, $X_t(f,k)$, where t indicates the time frame, f the filter bank channel and k the time vector in the matrix. The spectral-time matrix is then transformed into a cepstral-time matrix using a 2-D DCT. Since a 2-D DCT can be divided into two 1-D DCTs, an alternative implementation of the cepstral-time matrix is to apply a 1-D DCT along the time axis of a matrix consisting of M conventional MFCC vectors.

In the cepstral-time matrix, the lower index coefficients along the axis, n , represents the spectral envelope, whereas the higher coefficients represent the pitch and excitation, as is the case for conventional MFCCs. Along the axis, m , the lower coefficients represent the longer time variation of the cepstral coefficients, and the higher coefficients the short time variation. The column, $m = 0$, represents the average or steady-state of the spectral-time matrix. Figure 3 shows these regions.

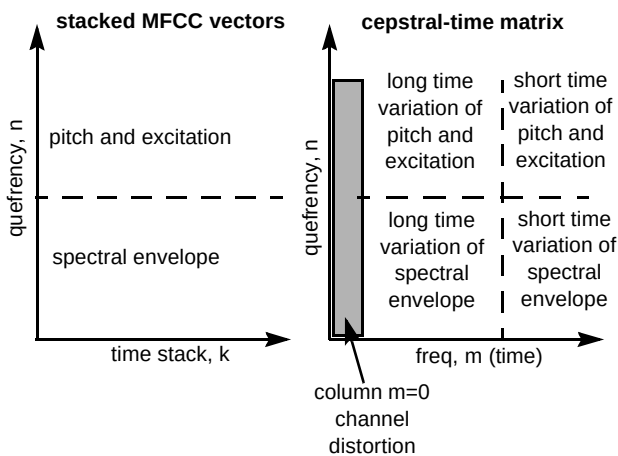


Fig 3 Regions of the cepstral-time matrix.

Only a sub-matrix of the cepstral-time matrix is useful for speech recognition, thus the matrix can be truncated down to $N' \times M'$. Additionally, if the steady-state column, $m = 0$, of the matrix is also discarded, any distortion of the speech signal by a non-ideal channel will be removed, making the cepstral-time matrix a channel robust feature.

2.3.1 Relationship between the cepstral-time matrix and differential parameters

Contained within the cepstral-time matrix is information regarding the transitional dynamics of the speech, as well as the instantaneous values. The normal way for including speech dynamics is to augment the instantaneous cepstral vector with differential parameters. A comparison can be made between the first- and second-order cepstral derivatives and the columns of the cepstral-time matrix. Equations (1) and (2) show that the cepstral derivatives are produced by weighted summations. As described, the cepstral-time matrix can be obtained by applying a 1-D DCT along the time axis of a matrix containing M MFCC vectors. The first and second columns of the cepstral-time matrix, $c(n,1)$ and $c(n,2)$, are given as:

$$c(n, 1) = \frac{2}{M} \sum_{k=0}^{M-1} c_k(n) \cos \frac{(2k+1)\pi}{2M} \quad \text{..... (3)}$$

$$c(n, 2) = \frac{2}{M} \sum_{k=0}^{M-1} c_k(n) \cos \frac{(2k+1)2\pi}{2M} \quad \text{..... (4)}$$

where $c_k(n)$ is the n th coefficient of the k th MFCC vector in the matrix containing the stacked MFCCs. Thus it can be seen that the first-order derivative and the $c(n,1)$ column of the cepstral-time matrix are both produced by a weighted summation of the static cepstral vectors, $c(n)$. Substituting equation (2) into equation (1) shows that the second order cepstral derivative is generated in a similar manner to that of the $c(n,2)$ column of the cepstral-time matrix. Figure 4 illustrates the similarity of the basis functions for producing the differential cepstra and the first and second columns of the cepstral-time matrix.

The similarity of the basis functions of the DCT and the differential cepstra can also be extended to higher orders [5]. It is interesting to note that for higher-order cepstral derivatives, larger time series of cepstral vectors are required. However, with the cepstral-time matrix the number of cepstral vectors in the stack remains constant.

Additionally, speech transitional dynamics are produced implicitly within the cepstral-time matrix, compared to the explicit representation achieved with a cepstral vector augmented by derivatives. Thus, models trained on cepstral-time matrices have the advantage that inverse transforms

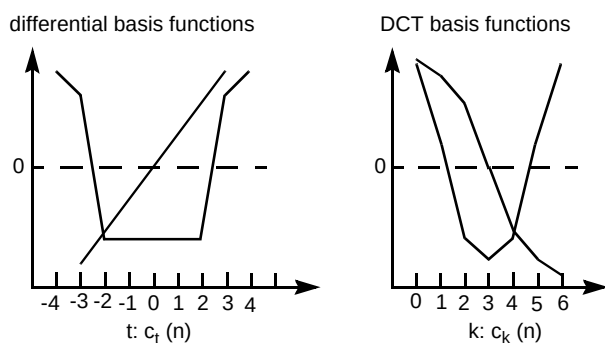


Fig 4 Differential and DCT basis functions.

can be applied which allow transforms back into the linear filter bank domain for such techniques as parallel model combination (PMC) [6].

2.4 Experimental results

The experimental results shown in this section demonstrate the improvements which can be gained by including temporal information into the speech feature. To examine the recognition accuracy of the various feature types a telephony-based speaker-independent isolated-digit task was chosen. Standard six state, seven mode, diagonal covariance HMMs were used to model the speech. Table 1 shows experimental results for a number of different feature types.

Table 1 Comparison of various feature types.

Feature type	Accuracy
8 MFCC	77.3%
8 MFCC + 8 Δ MFCC	91.0%
9 $\times$ 4 CTM (a)	94.8%
9 $\times$ 3 CTM (b)	96.9%
8 $\times$ 4 CTM (c)	92.6%
8 $\times$ 3 CTM (d)	94.4%
9 $\times$ 2 CTM (e)	96.7%

The most obvious conclusion from these results is that including temporal information on to standard MFCCs improves the performance from 77.3 to 91.0%. Table 1 also shows performance for cepstral-time matrices which implicitly include temporal information. Figure 5 illustrates the regions of the cepstral-time matrix that have been used in each experiment.

The results show that best performance is obtained when a 9 $\times$ 3 cepstral-time matrix is used. This matrix effectively contains first-, second- and third-order time derivatives of the cepstrum, with no instantaneous cepstrum. It is interesting to observe that removing the third

column of the matrix only reduces accuracy from 96.9 to 96.7%, but reduces computation by a third. Higher accuracy is achieved when the zeroth column of the cepstral-time matrix is removed, i.e. compare 9 $\times$ 3 with 9 $\times$ 4 and 8 $\times$ 3 with 8 $\times$ 4. Contained within this column is the steady-state component of the MFCCs, which includes the cepstrum of the channel.

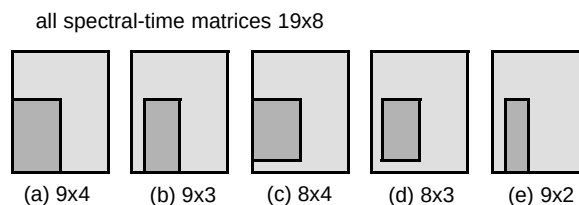
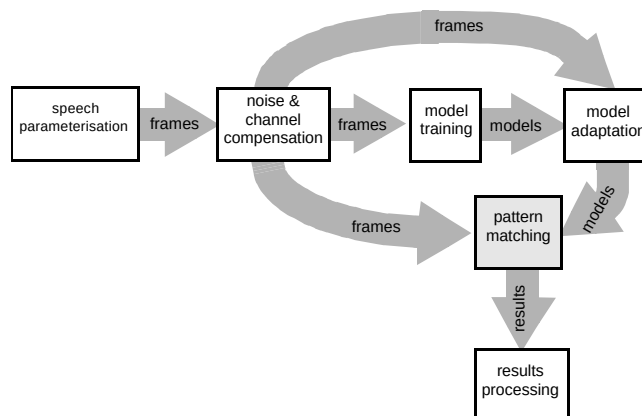


Fig 5 Spectral-time to cepstral-time truncations.

2.5 Discussion

Many improvements can be made to conventional cepstral features to improve their performance. Perhaps the most important of these is to include some temporal information concerning the way that speech changes in time. This helps to overcome the limiting IID assumption of HMMs and can improve recognition performance. A simple way of including this information is to explicitly augment time derivatives of the cepstrum on to the instantaneous cepstrum, which has been shown to give a substantial improvement in recognition performance. A mathematically more implicit method of including temporal information is to use cepstral-time matrices. These are shown to give considerably higher recognition accuracy compared with conventional cepstra augmented with higher order derivatives.

3. The pattern-matching process



Once the incoming speech signal has been parameterised, the actual recognition process involves trying to match it to some word or set of words within a predefined vocabulary. Hidden Markov models representing words or sub-word units within the vocabulary can be combined as a

network, and the problem then becomes that of finding the path through that network that best matches the signal in question.

Modern tasks often involve very large vocabularies, comprising many thousands of words and high perplexities¹. In order to reduce the computational requirements of these tasks to a level where real time performance is feasible, a new generation of recognisers have been developed which implement a two-pronged speed-up strategy.

The first tactic employed is to use as many *a priori* knowledge sources as possible. Such sources typically include the implementation of statistical grammar, where the probability of a given word following a previous set of words is known, or a finite state grammar, where knowledge of syntax and semantics is taken into account.

The second tactic is to reduce the amount of network in use at any given time. One established route that helps achieve this is the use of a phoneme look-ahead or fast preliminary network pass in order to reduce search space. A more recent technique is to move from a **static** network recogniser to a **dynamic** network recogniser [7]. It is this approach which is discussed in this section.

Static network recognisers load a predefined network at the commencement of recognition. This network describes fully all possible utterances that can be recognised. A dynamic network recogniser, however, creates a network during recognition. At any given time, the network can be extended where necessary by adding extra words or phrases. Similarly, those parts of the network not in use can be disassembled and need no longer be considered. This approach ensures that only the minimum amount of network is ever in use.

Unavoidably, a significant amount of processing is involved in the dynamic creation and destruction of the network, and this can result in slower recognition times for small tasks. When applied to long utterances and large vocabulary tasks, the cost of dynamic network creation becomes trivial compared with the memory savings made.

3.1 System architecture

Figure 6 presents a typical block diagram of a modular dynamic network recogniser. The architecture is designed around a central network, which provides a repository for all information about the system. As mentioned in the previous section, a network comprises of a lattice of nodes, each of which has an associated HMM representing a word or sub-word unit. Nodes may also contain other information, such as a word name, phoneme name and language model penalty score. Other modules are able to operate upon the network using the information found therein in order to produce a series of word hypotheses with

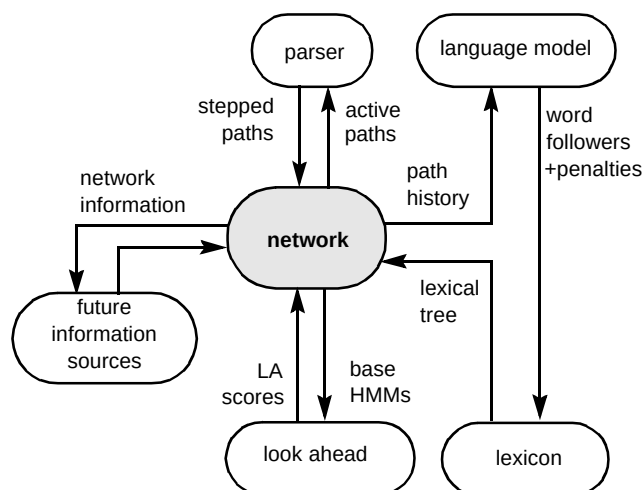


Fig 6 System architecture.

associated probabilities when attempting to recognise an utterance. The operation of each of these modules is, ideally, independent of the others, and a given recogniser need only contain a subset of the modules shown. The advantage of a modular design such as this is the ease with which subsequent additions to the system can be incorporated.

3.2 The parser

The core module used in all recognisers, both static and dynamic, is the parser. Typically the parser performs a conventional time-synchronous Viterbi beam search on the network [8] constrained by a dynamic pruning algorithm. During recognition, this generates a set of partial path hypotheses, each with an associated likelihood, or 'score', based on the acoustic match between the models in the partial path and the incomplete utterance. These partial paths are extended on a frame-by-frame basis, and those with scores that fall below an acceptable threshold are removed by the pruning algorithm.

3.3 The language model

As observed at the beginning of section 3, the language models used are usually either statistical or finite state. An n -gram statistical language model is one which contains information about the probability of any word occurring after a string of $n-1$ previous words. The problem with this approach is that there is a vast number of n -grams — for a k word vocabulary there are k^n n -grams. Even for the most widely used bigram (2-gram) language models, a 5000 word vocabulary implies a potential 25 million bigrams. As it is clear that even the largest training sets are unlikely to contain examples of all of these, a compromise frequently adopted is to use a back-off language model. An n -gram back-off language model copes with the absence of statistics for a given length n word string by backing off to a modified $(n-1)$ -gram probability for that word string [9].

¹ In this paper perplexity is defined as the average number of nodes fanning out from a given node, i.e. the average network branching factor.

This process can be repeated, if necessary, until simple unigram (1-gram) weights are used.

A finite-state grammar restricts the choice of words following a given string of previous words. The list of allowed words can be dictated by semantic or syntactic considerations. In a dynamic network recogniser, the string of previous words typically corresponds to some partial path through the network, and the list of following words encompasses all the possible ways this path might be extended.

3.4 The lexicon

To use the list of words generated by the language model, it is necessary to create a small fragment of network representing those words. One way of achieving this is to use a lexicon, which firstly develops phonemic representations of the list of following words, and then, from these representations, is able to generate a set of sub-word models representing the words. Finally, this set of sub-word models can be attached to the end of the partial path, ready for the next frame of data. Together, the language model and the lexicon thus provide the capability to extend a network dynamically during the recognition process.

An example of this is shown in Fig 7. Figure 7 (a) shows a partial path comprising the phoneme models making up the words 'the' and 'cat'. Figure 7 (b) gives an example of the set of allowed following words that might be returned by a call to a language model. In Fig 7 (c), a network fragment derived from this set of words is shown. Note that in this example, the lexicon has created this fragment in the form of a tree. This enables different words with the same initial sequence of phonemes to share the same models, and thus reduces the size and perplexity of the network. Finally, in Fig 7 (d) the dynamic network extension process is completed by attaching the new network fragment to the end of the partial path.

3.5 Predictive strategies

The purpose of any predictive strategy is to reduce the number of active states by performing a quick rough, or fast match upon all current hypotheses to ascertain their likelihood. Only those which match reasonably well to the acoustic signal need be considered for the subsequent detailed match. Such strategies can broadly be divided into two categories — those that use a two-pass search algorithm, and those that use a phoneme look-ahead.

Two-pass search algorithms typically define the network with a fast first pass, and then do a detailed match constrained by that network. Because the first pass can produce rough estimates of path scores, a popular method is to reverse the direction of the second pass, and use an A*

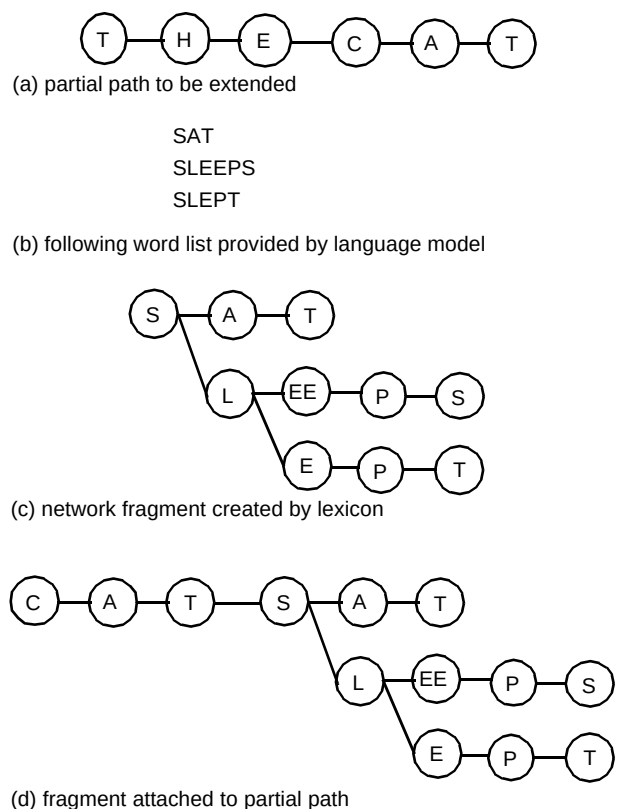


Fig 7 Dynamic network extension example.

stack decoding algorithm to extend partial hypotheses in a non-time synchronous fashion [10].

Phoneme look-ahead algorithms reduce the number of active states by estimating how well a particular phoneme matches the next few time frames. At a given time frame, only those paths about to be extended into relatively likely phonemes need be considered [11].

3.6 Path merging

The above technique results in a tree structure with numerous branches that differ only slightly. Clearly a considerable saving could be made if similar branches could be merged together. However, if path merging is implemented without due consideration, the quality of the top-*N* output of an *N*-best¹ recogniser can be degraded.

For the purposes of discussion, path merging can be divided into three approximate levels:

- instant path merging,
- merging paths with similar recent histories,
- no path merging.

¹ One in which the *N*-best scoring results are output at the end of recognition.

In the case of instant path merging, all paths that reach a word boundary are propagated into a single set of following models. Only the best partial hypothesis up to the merge point is continued, and all other partial paths are stored for subsequent use in traceback. Although the propagation of the top path is guaranteed, the final N -best output may be sub-optimal, as other path hypotheses can compete directly with (and hence be destroyed by) the winning hypothesis.

The root of the problem observed above lies in the potential for alternate hypotheses to have different time segmentation at the merge point. Optimal N -best is assured if differing hypotheses are only merged when their segmentation has become aligned. Unfortunately, there is no hard and fast way of ascertaining this. An interim step towards establishing the point at which two partial hypotheses may become time aligned is to split the history of each partial hypothesis into **distant** and **recent** history. Recent path history is defined such that the segmentation of two partial hypotheses which share the same recent path history will have become time aligned by the end of that recent history.

From this definition, it is evident that the merging of only those paths with identical recent histories will ensure the optimal N -best output of a recogniser. The remaining difficulty is to establish exactly the length of the recent history. Schwartz and Austin [12] suggest using a 1-word recent history, with encouraging results, although the variation in word length may cause some errors.

In practice, this minimum length is often more than satisfied due to constraints imposed by the language model. In section 3.3, it was observed that the language model of choice is often an n -gram statistical one. When implementing path merging in conjunction with an n -gram language model, the recent history of hypotheses must also equal or exceed the span of the language model ($N-1$ words) before path merging between two hypotheses can take place. This ensures matching penalties on the resulting node tree shared by both hypotheses.

3.7 Integration into spoken dialogue systems

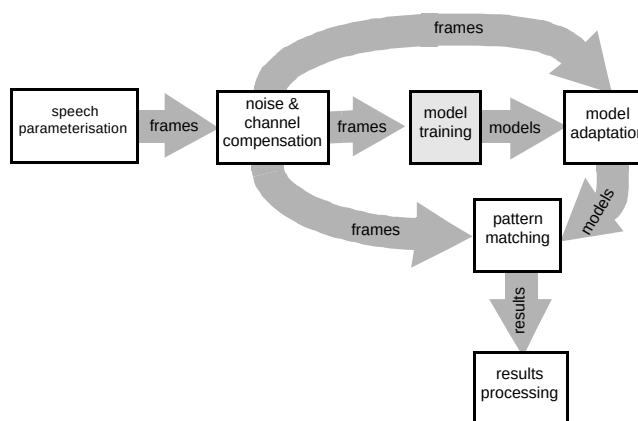
The output of a recognition pass is an acyclic graph of word and phrase hypotheses. This can be searched efficiently (using, for instance an A* algorithm) to extract a top- N candidate list, or can be passed in its entirety to a speech understanding unit as part of a spoken dialogue system.

When employed as part of such a system, the flexibility of a dynamic network recogniser comes to the fore. The need to switch between domains, or re-recognise a particular utterance using tighter constraints (to ascertain a place-name, for example) is achieved simply by employing — and, if necessary, creating — the appropriate language

model, rather than having to load up a specific predefined network exclusively for one recognition pass

The use of a dynamic network recogniser thus presents an efficient solution to dealing with large vocabularies or long utterances. Several strategies can be employed to minimise the size of network in existence at any given time. In addition, dynamic recognisers can be integrated easily and efficiently into natural language dialogue systems.

4. Model training



Recognition system vocabularies can either be based on whole-word or sub-word models. Whole-word models are derived from several training examples, covering all possible variations. Although this approach is fairly straightforward, and generally yields better results than sub-word models, it is limited by the fact that each vocabulary item requires a separate model and separate training data.

Vocabularies based on sub-word models use a different approach. Individual vocabulary items are formed from combinations of phoneme models. If it is assumed that the phonemes are context independent, then only 44 models, corresponding to the 44 phonemes in the standard British English phoneme alphabet, are required. In practice this is not a very good assumption since phonemes can be highly dependent upon context.

Contextual effects may be accommodated by moving to a triphone-based approach. The transition from a context-independent to a context-dependent modelling system inevitably results in an increase in the number of models to be trained. Consequently the amount of training data afforded to each model can become grossly inadequate.

The problem of insufficient training data may be overcome by using tied parameter models. The following section considers a method of relating the structure of tied-parameter context-dependent systems to the amount of available training data and to the variability within it. Three separate tied-parameter systems are compared. The systems

are shown to be more accurate than their uniformly structured counterparts.

The trade-off between model specificity and model trainability has been the subject of much research for a number of years [13–15]. One trend has been to develop context-dependent HMM sub-word unit systems which compensate for the lack of allophone ¹ training data by tying the parameters of each group of allophones, associated with the same monophone ², to a separate pool [16, 17]. Systems which keep the number of shared model parameters constant across groups of tied allophones fail to address two important aspects of the training problem. Firstly, there is typically a large variance in the distribution of training data across the allophone groups and secondly the variability of sounds to be modelled within each group is different.

The approach presented here aims to link the number of parameters shared by each group of models to the trainability of that model group and also to the variability within the group.

The parameter pool size is calculated via the ‘rule of thumb’ given by equation (5). Trainability is defined as the number of training tokens available for the allophone group and variability is defined as the number of different triphone contexts present within the group:

$$S_m = F \times \sqrt{T_m} \times V_m \quad \text{..... (5)}$$

where:

S_m is the parameter pool size for allophone group m ,

F is the constant which is used to control the total number of parameters,

T_m is the number of training tokens associated with allophone group m ,

V_m is the number of triphone contexts present in group m (measure of variability within group).

This equation has been applied to two tied-parameter systems. The task used to evaluate the performance of this technique was the speaker-independent recognition of examples of 697 different surnames collected over the UK telephone network (see section 4.4).

4.1 Context-independent seed models

Sets of context-independent models are required as seed models for the tied-parameter systems and also to provide baseline context-independent performance levels. The context-independent symbol set used for this work comprises 44 phoneme symbols and one noise symbol. Two main systems were created based on this symbol set.

¹ One of two or more forms of the same phoneme.

² Context-independent realisation of a phoneme.

Model Type 1 was a 3-state 12-mode system containing 1584 modes in total.

Model Type 2 was a 3-state variable-mode system also with a total of 1584 modes. This system was created by applying equation (5) during the allocation of modes to each state of each context-independent unit. In this case each context-independent unit is treated as an allophone group.

The performance of the two systems is presented in Table 2 in section 4.5.

4.2 Tied-parameter systems

The following sections describe the three types of tied-parameter system investigated.

4.2.1 Shared-mode system (SMS)

In this system the corresponding states of all the allophones, derived from the same monophone, share a common pool of modes. The application of equation (5) to this system results in a variation in the size of the shared mode pool from one allophone group to the next. Figure 8 shows the arrangement for a single allophone group where the corresponding states in each allophone share a pool of three modes. Allophones within a group differ only in their mode weights.

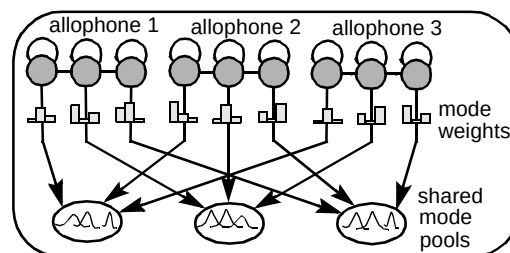


Fig 8 Shared-mode system.

4.2.2 Shared-mode and clustered-state system (SM&CSS)

All the allophones derived from the same monophone share a common pool of modes and a common pool of states for corresponding states. Figure 9 shows the arrangement

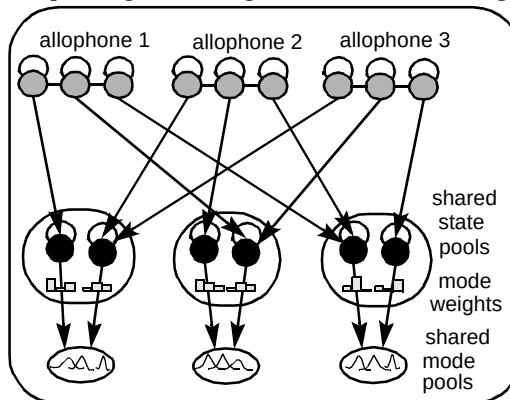


Fig 9 Shared-mode and clustered-state system

Fig 9 Shared-mode and clustered-state system.

for a single allophone group where the number of corresponding states has been reduced from 3 to 2 by the state clustering process, and the states in each pool share a pool of three modes. Comparing Fig 9 to Fig 8 it can be seen that the two systems are essentially the same except that the shared mode and clustered state system has fewer sets of mode weights. State clustering applied to a shared mode system effectively results in the clustering and sharing of sets of mode weights whilst the modes themselves remain unaltered.

4.3 Triphone construction

The recognition network for this task contains 2210 different triphones. Of these 23% occur three or fewer times in the training corpus and only 14% have 50 or more occurrences. For this reason it was decided that for each experiment two shared-parameter systems would be constructed, one for left biphones and one for right biphones. Following the re-estimation of both shared-parameter systems the triphones required for recognition are synthesised from the appropriate biphones.

4.4 Database

The training corpus used for these experiments consisted of 13 448 examples of isolated surnames collected over the UK telephone network from callers spread throughout the country. There were in total 8000 different surnames in the training corpus. The test data was a separate set comprising 494 examples of surnames collected in the same manner. There were in total 697 different surnames in the recognition network. Some of the surnames had multiple pronunciation transcriptions resulting in a network size of 1000. The 95% confidence interval for this task is 4.3% based on an assumed error rate of 58%. A standard cepstral-based signal parameterisation was used.

4.5 Experiments

To compare each of the shared parameter systems, the total number of modes in each system was kept constant and equal to the number of modes in the context-independent systems described in section 4.1. Although the number of modes is kept constant the number of mode-weights varies from system to system. All the recognition times quoted are relative to that of the standard fixed-mode context-independent system shown in row 1 of Table 2.

The timing information was acquired by applying the UNIX command time to each run of the HTK recognition tool HVite [18]. All vocabulary models are 3 state left to right with no skips. A single-state 6-mode noise model was used throughout.

Table 2 Context-independent results.

Model type	Percentage accuracy	Recognition time
1	54.7	1.0
2	56.3	0.8

4.5.1 Context-independent experiments

For comparison with the context-dependent systems Table 2 shows the results obtained by a 3-state 12-mode context-independent system (model type 1) and also those achieved by a 3-state variable-mode context-independent system (model type 2) containing the same total number of modes.

4.5.2 Shared-mode system experiments

Using the context-independent models as seed models two separate context-dependent, shared-mode model sets were created:

- set 1 — left and right biphone SMSs generated using the fixed-mode context-independent models as seed models,
- set 2 — left and right biphone SMSs generated using the variable-mode context-independent models as seed models.

The SMS were then re-estimated and the triphones required for recognition were synthesised from the two biphone SMSs. The results for sets 1 and 2 are shown in Table 3.

Table 3 Context-dependent SMS results.

Set	Percentage accuracy	Recognition time
1	58.5	2.8
2	60.9	3.2

4.5.3 Shared-mode system and clustered-state system experiments

State clustering was applied to the left and right biphone SMSs generated as set 2 in the previous experiment and three separate context-dependent, shared-mode and clustered-state model sets were created:

- set 3 — the number of states in each system was reduced by 25%,
- set 4 — the number of states in each system was reduced by 50%,
- set 5 — the number of states in each system was reduced by 75%.

The SMS and CSS were then re-estimated and the triphones required for recognition were synthesised from the two biphone SMS and CSSs. The results for sets 3, 4 and 5 are shown in Table 4.

Table 4 SMS and CSS results.

Set	Percentage accuracy	Recognition time
3	61.3	1.8
4	59.7	1.7
5	58.5	1.5

4.6 Discussion

There are some advantages to be gained in modelling contextual variation within sounds, especially when this technique is coupled with a modelling structure which reflects the amount of training data available and the variability within the sounds to be modelled.

This work also demonstrates that it is possible to produce accuracy improvements without the computational overhead normally incurred with context-dependent modelling.

Compared with fixed-mode context-independent modelling, the two shared-parameter systems produced differing performance in terms of both recognition accuracy and computational efficiency.

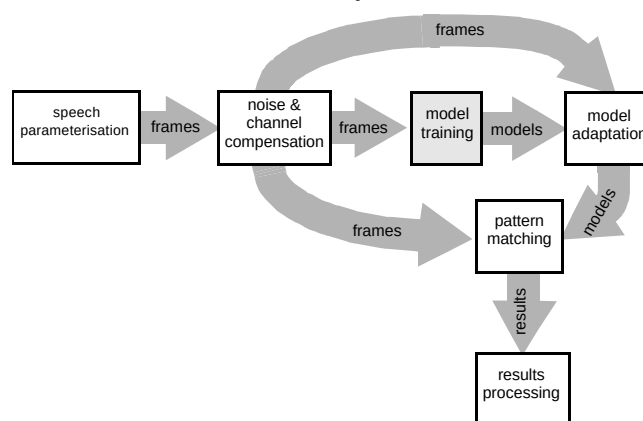
Shared-mode systems were found to give gains in terms of recognition accuracy at the cost of increased computation at recognition time. Recognition accuracy rises by 6.2% (from 54.7% to 60.9%) with a corresponding rise of 220% in recognition processing time.

Shared-mode, clustered-state systems succeed in dramatically reducing the computational overhead of the shared-mode systems while maintaining the accuracy gains achieved. Recognition accuracy rises by 6.6% (from 54.7% to 61.3%) with a corresponding rise of 80% in recognition processing time.

Taking both recognition accuracy and processing time into consideration it would appear that the shared-mode, clustered-state system provides the best overall performance.

It was mentioned in the introduction that lack of training data and model inflexibility are two factors that determine how well the contextual variations within a phoneme can be modelled. This work has shown that variable pool size shared-parameter systems provide increased model flexibility, allowing the models to make more efficient use of the limited training data available.

5. User-selectable vocabulary



In addition to system defined vocabularies there are situations where the end users choose their own vocabularies and hence where the vocabularies cannot be trained in advance. The most common application where this situation arises is in ‘voice dialling’ where the user just speaks the name of the person they wish to call and the system then places the call. Clearly it is impossible to predict in advance the choice of names that each user will select, so the system must be trained by each user to recognise their particular selection of names. This type of recognition is termed ‘user-selectable vocabulary’.

The traditional approach to user-selectable vocabulary recognition has been to use speaker-dependent whole-word modelling techniques. In an enrolment session the user is asked to say each of the words one or more times and from these examples a model or template is created for each word. Dynamic time warping (DTW) against these templates is the technique then typically used to perform the subsequent recognition.

An alternative approach is presented here which employs sub-word HMM techniques. The principle behind the new technique is to generate phonetic transcriptions of the user’s words and then create a recognition network which allows only those phoneme sequences seen in the enrolment utterances. The sub-word technique is described below and the results of experiments comparing it with a DTW whole-word system are presented.

5.1 Theory

The principles behind a sub-word HMM-based user-selectable vocabulary system are relatively simple. The first step is to generate one or more sub-word transcriptions for each of the words in the vocabulary. This can be done by presenting the utterances of the user to a sub-word HMM recogniser with a grammar that imposes little or no constraint on the possible sequences of sub-word units recognised. The transcriptions generated can then be used to

define a recognition grammar which will allow only the words that the user has chosen.

5.1.1 Transcription generation

In the simplest case, the words that a user speaks may be transcribed using a sub-word HMM recogniser that imposes no grammar constraints whatsoever. In this case phonemes have been used as sub-word units and modelled with HMMs, trained in a speaker-independent manner. In practice, however, the accuracy of an unconstrained phoneme recogniser operating on telephony speech is currently not very high. Furthermore, unconstrained recognisers are prone to phoneme-insertion errors in 'silence' periods (which often are not very silent). Consequently, the approach adopted here was to impose a weak grammar upon the sub-word recogniser. Figure 10 shows the form of the grammar used. This grammar allows only a single connected sequence of phonemes surrounded by noise.

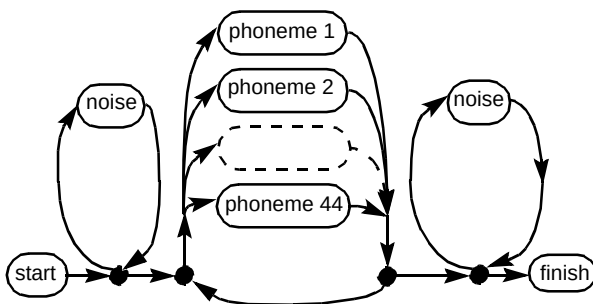


Fig 10 Network used for generation of phonemic transcriptions.

Due to the weakness of the grammar, the recogniser can generate phoneme sequences that do not occur in the English language. To improve the transcriptions generated it is possible to impose a stronger grammar that restricts the possible phoneme sequences. One possibility is to use a phoneme bigram grammar that constrains which phonemes may follow which others. The example in Fig 11 shows the sort of transcriptions that the system generates for a few spoken digits.

Without bigrams	With bigrams
one = W AA MA	one = W AA M A
two = F B S UU A	two = T UU A
three = TH R EE	three = TH R EE
four = V V V F AW E	four = B V F AW
five = F AR AI A TH	five = F AR AI A TH
six = S AI K S	six = S I K S

Fig 11 Examples of system-generated transcriptions.

Figure 11 shows that the transcriptions generated using the phoneme bigram restrictions look more plausible than those from the totally unconstrained network.

5.1.2 Recognition network generation

When one or more transcriptions have been generated for each of the words in the desired vocabulary, a recognition network may be generated. This is a straightforward process which may simply combine the individual transcriptions into parallel alternatives in the network or, more efficiently, into a tree network.

In a practical system it may be desirable to request several examples of each word from the user. These can then be checked to ensure that the transcriptions generated are at least reasonably similar to each other, thus providing a measure of robustness against bad enrolment utterances. They can be further checked to ensure that they are significantly different from those for other words, thus protecting the user from selecting a vocabulary which is too difficult for the recogniser. Figure 12 shows an example of a recognition network generated from automatically generated transcriptions of the words 'one', 'two' and 'three'.

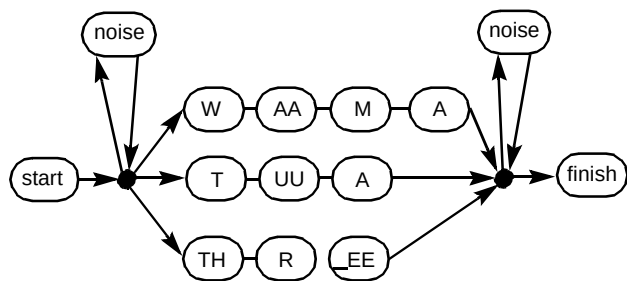


Fig 12 Example of a network generated from transcriptions of utterances of the words 'one'.

5.1.3 Advantages of the sub-word technique

The sub-word method has a number of advantages over the conventional whole word techniques.

It lends itself easily to mixing predefined words (which need no user training) with user-selected ones because the sub-word models are speaker independent. For example, a voice-dialling application developer could predefine the words 'call', 'help' and 'exit' and, if the user then enrolled with the names 'Mrs Jones' and 'doctor', a recogniser could be constructed that would allow the user to say 'call Mrs Jones', 'call doctor', 'help' or 'exit' even though the user had not previously given any training examples of the predefined words. This feature also gives the application developer the flexibility to add new control words after the service is up and running without requiring any further enrolment data from the users. This would be much more difficult with the whole-word approach as it would either require the mixing of speaker-dependent and speaker-independent models (which might have a detrimental effect on accuracy) or require the user to provide enrolment examples of all the predefined words.

The sub-word technique also requires over an order of magnitude less storage than the conventional technique to store the vocabulary of each user. Sub-word uses only about 10 bytes per word to store the transcription rather than a few hundred or thousand bytes to store a model or template.

5.2 Experiments

Experiments were carried out to compare the sub-word HMM technique with a more conventional whole word DTW system.

A 14-word test vocabulary was chosen, consisting of the digits 'one' to 'nine', 'oh', 'zero', 'nought' plus two control words, 'highline' and 'chequebook'. The data was taken from the BT Brent telephony database. For each of 100 talkers, between one and three repetitions of each word were used for enrolment, each repetition being taken from a separate phone call. The test set consisted of seven repetitions of each word, again each taken from a separate phone call, where the calls were placed over a six month period.

The results in Table 5 show that the conventional DTW system is 1—2% more accurate than the sub-word system, but much more significant is the effect of the number of training examples used.

Table 5 Comparison of sub-word HMM and HTW systems.

Recogniser	Number of training examples per word	Accuracy%
sub-word HMM	1	88.8
sub-word HMM	3	94.1
whole word DTW	1	90.8
whole word DTW	3	95.3

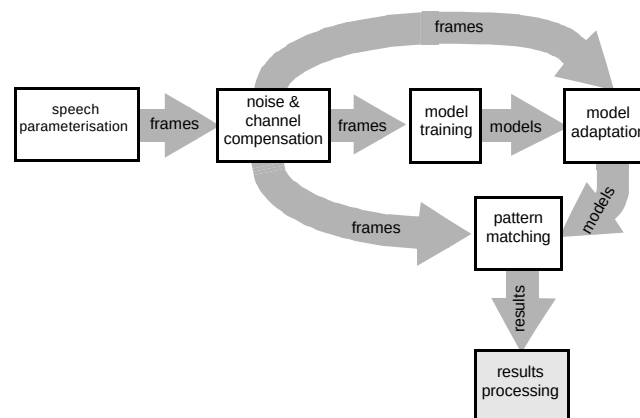
A second set of experiments were performed to see how well the sub-word HMM system would fare with vocabularies that were part user-trained and part predefined. This meant replacing transcriptions generated from the user's enrolment utterances with 'idealised' pronunciations for a proportion of each user's vocabulary.

The results in Table 6 show that use of predefined words reduces the accuracy to about the same as that obtained when only a single training example is used. This observation lends support to conclusions reported elsewhere [19] that for speaker-independent recognition, performance can be improved if the lexicon of standard pronunciations is supplemented with automatically generated transcriptions of real utterances.

Table 6 Performance of sub-word HMM system when part of the vocabulary is predefined.

% vocabulary pre-defined	Number of training examples per word	Accuracy%
0	1	88.8
0	3	94.1
50	1	88.3
50	3	89.3
100	0	88.2

6. Out-of-vocabulary rejection



In automatic speech recognition, one of the most difficult conditions to reliably detect is when the input speech contains a word not in the recogniser vocabulary. The problem of out-of-vocabulary (OOV) rejection remains a challenging one, crucial to the robustness of commercial recognisers. An OOV condition occurs when an input speech signal cannot be correctly decoded into any of the possible recogniser outputs. This can happen in several ways. For example, the spoken input to a 'yes/no' (isolated word) recogniser may be an utterance of the word 'banana'. Alternatively, a spoken digit sequence may be input to a connected digit recogniser which is not in the expected set of digit sequences. One solution is to attempt to recognise all possible spoken input. Clearly this method would be prohibitive in terms of the computation and storage requirements.

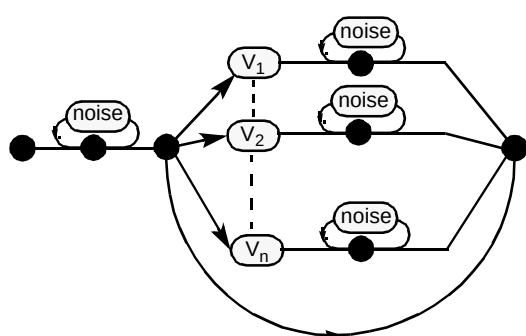
A more conventional approach is to process the model scores after the recognition stage. One would expect a poorer score for speech not corresponding to a model than for a valid match to the model. A prescribed threshold, applied to each model score, would enable discrimination between vocabulary and OOV elements of the input speech. Score post-processing turns out to be a surprisingly poor arbiter of OOV occurrences (see section 6.1).

The solution proposed here operates at the level of acoustic pattern matching. Generic models of speech, or

sink models, are incorporated into the recognition process to detect extraneous speech. The issues involved in training sink models and incorporating them into the recognition framework are discussed in section 6.2. This technique dramatically improves OOV performance but at greater computational cost compared with score post-processing.

6.1 Score post-processing

An example grammar network for an isolated-word recogniser is shown in Figure 13. This network includes a noise-only, or silence, path. Any input to this network can only be 'recognised' as one of vocabulary models V_1 to V_n , or silence. OOV occurrences can only be detected by processing information arriving at the output node, e.g. path scores or model traceback data.



V = vocabulary model

Fig 13 Grammar network for n parallel vocabulary models.

Score-based rejection is performed by the two following tests:

$$\text{reject if } S_{\text{best path}} < t \quad \text{or} \quad \frac{S_{\text{best path}}}{S_{\text{next best path}}} > t,$$

where $S_{\text{best path}}$, $S_{\text{next best path}}$ are the best and next best path scores respectively and t is a threshold. Varying t allows the rejection rate to be controlled.

One would expect the best path score to be poor for OOV speech input. This is tested by the first condition. The second condition examines the score ratio between the best and second best paths. This detects when the best path is not a clear winner which may suggest the data is not matching particularly well to any of the models.

The effect of applying score-based rejection techniques to an isolated-digit vocabulary is illustrated in Fig 14. The vocabulary consists of 13 words: digits 'one' to 'nine', 'zero', 'nought', 'oh' and 'stop'. The test set consists of 1209 spoken digit examples. The OOV test set consists of 369 spoken town name examples. This set contains 143 different OOV words making a good OOV test set, as the

OOV detection performance is not likely to be biased towards a particular word.

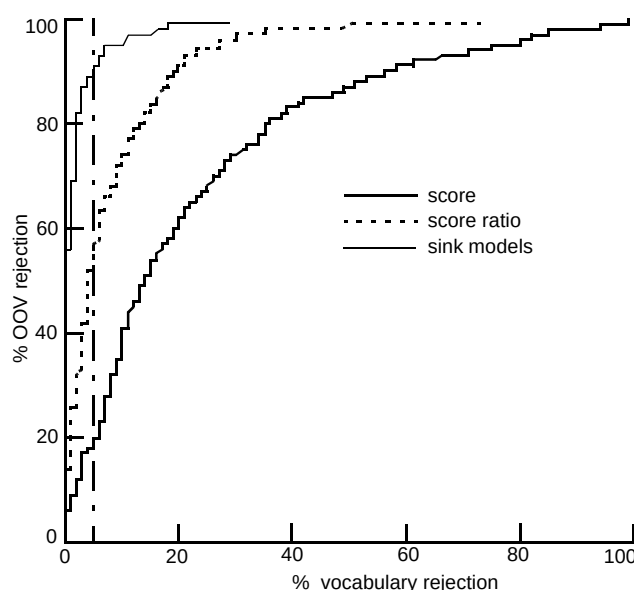


Fig 14 OOV rejection performance curves.

As can be seen from Fig 14, rejecting any OOV words incurs a cost of rejecting correctly recognised words. To compare the different rejection techniques a comparison point of 5% vocabulary rejections — denoted by the vertical dashed line — is shown. Anything greater than this is usually unacceptable in a real system.

At 5% vocabulary rejection, the score and score ratio reject about 20% and 55% of OOV words respectively. This poor performance may be attributed to the simplifying assumptions inherent in the modelling of speech signals with standard continuous density hidden Markov models (CDHMMs). Furthermore, the need to train models over a wide variety of speakers and channels for telephony applications greatly increases the variability of valid speech signals matching to a particular model. The poor performance of the score-based classifiers suggests a large overlap between model scores for valid matches and model scores for OOV matches.

6.2 Sink models

As noted earlier it is not feasible to attempt to recognise all possible OOV inputs. Even if recognition accuracy permitted, the computational overhead would be prohibitive. A suitable compromise is to use sink models as generic models of OOV speech.

Sink models are used in parallel with vocabulary models in a network to provide alternative hypotheses at a decision node. The original network, with parallel sink models

added, is shown in Fig 15. Speech is classified at the exit node as follows:

$$\text{reject if } \frac{S_{\text{vocabulary}}}{S_{\text{sink}}} > t$$

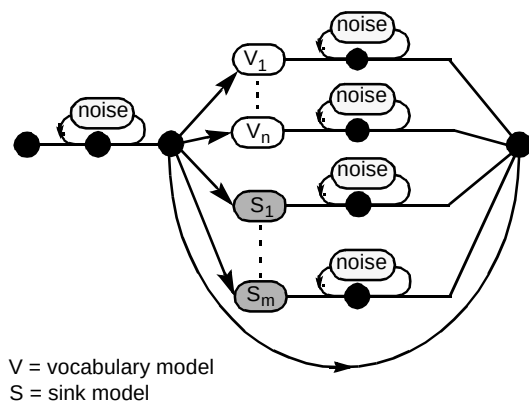


Fig 15 Grammar network including m parallel sink models in distinct parallel paths.

where $S_{\text{vocabulary}}$ is the best scoring vocabulary path, S_{sink} the best scoring sink path and t is the rejection threshold. The rejection rate is again controlled by varying t .

Sink models are not intended to model a specific sound. The approach to training is to train on many utterances (as for vocabulary models) and also on a wide variety of different speech sounds. The effect of this is to widen the state probability densities compared with the vocabulary models. This is shown in Fig 16 for a single feature.

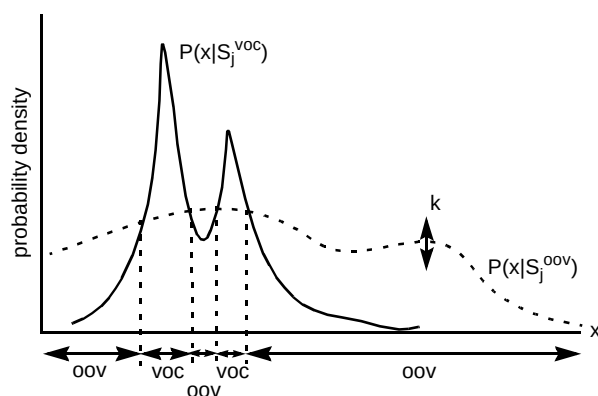


Fig 16 Example pdfs for state j of a competing vocabulary and sink model.

If the speech data (represented by feature x) consistently occupies a region where the pdf for state j of the sink model $P(x|S_j^{\text{oov}})$ is greater than for state j of a vocabulary model $P(x|S_j^{\text{voc}})$, the sink model will tend to score better relative to the vocabulary models.

The choice of sink model topology is a contentious issue. In principle, a single state model with a self-loop should suffice as temporal structure is meaningless for such a generalised model.

Experimentation shows that multi-state sink models give better results. This is possibly because the sink paths accurately segment regions of speech as effectively as the vocabulary paths.

The effectiveness of the sink path at segmenting speech can be fine-tuned by weighting the sink model. This takes effect in the HMM forward probability calculation $\alpha(S_j, t)$ of being in state j of the sink model at time t :

$$\alpha(S_j, t) = \text{Max}_i \{k \cdot \alpha(S_i, t-1) a_{ij} P(\underline{X} | S_j)\} \quad \dots\dots (6)$$

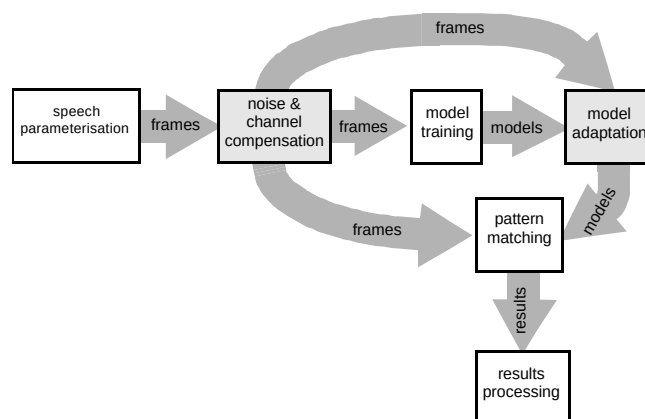
where a_{ij} is the transition probability from state i to j ; $P(\underline{X} | S_j)$ is the likelihood of observation $\underline{X}$ given state j and k is the model weighting factor. The weighting factor k effectively slides the sink model probability density up or down as shown in Fig 16.

The OOV rejection results from the same isolated-digit and OOV test sets are also shown in Fig 14. Two parallel sink models were used — one trained on 5558 words of sets A—B of BT's TADS surname database (4212 different words); the second on 1585 words of sets A—E of BT's LETRASET surname database (100 different words).

The performance gained from using the two sink-models is a dramatic improvement over score-based methods to ~90% OOV rejection at the 5% vocabulary rejection level.

Future improvements in modelling the acoustic signal are likely to complement the sink-model approach and lead to increased recogniser robustness.

7. Speech recognition in adverse conditions



Leading edge speech recognition systems are capable of highly accurate performance for particular tasks within well-defined operational environments. However, to fully exploit the technology, practical systems must be capable of

operating in the range of different environmental conditions encountered in everyday use. For example, a problem associated with the use of car phones and mobile equipment is that the background noise and communications channel response are likely to be highly volatile not only between calls, but potentially may change every few seconds of the call duration.

Although current designs tailored to a particular environment (known as 'matched' recognisers) will give good performance in the local ambient conditions, the performance of these dedicated systems falls off severely as background conditions move away from the training environment.

The problem is clearly illustrated in Fig 17, which presents the results of an experiment outlining the

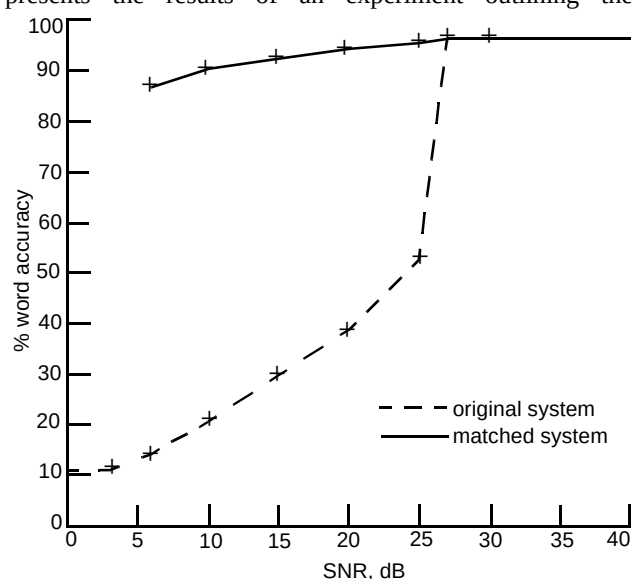


Fig 17 Recognition performance in noisy conditions.

performance of an HMM-based isolated digit recogniser trained in relatively noise-free conditions and tested with noisy utterances over a range of signal-to-noise ratios (SNRs) from 'clean' (>25 dB) to very noisy (~0 dB). As a comparison, a second experiment presents the matched (or 'optimum') recogniser performance at each point. For this experiment, sets of speech models were trained with data at each of the SNRs considered in the first experiment. The results of this experiment are also plotted in Fig 17.

The experiments demonstrate that good performance can be obtained in very noisy environments, if the speech models and testing environment remain appropriately balanced. It is the aim in this section to present several different techniques for achieving this balance, each leading to a more robust recognition system design.

7.1 Dealing with adverse conditions

A range of different techniques for accomplishing noise-robust speech recognition has been established. These may be divided into two separate groups. The first approach applies some form of preprocessing to the incoming speech to compensate for the interfering signal (such as filtering). The second attempts to modify the pattern-matching stage (for instance, by adapting the speech models to the new conditions).

The choice of compensation technique depends upon the range of adverse factors likely to be present in the operating environment of the recognition system. These fall into three basic categories — interfering background noise, communication channel effects (such as echo and distortion), and the altering of human speech production in challenging conditions. The last factor, known as the 'Lombard effect', is the most complicated of the three, as it is difficult to isolate the instantaneous changes in pitch, intensity and spectral shape which result from the 'stressed' speech production. Compensation methods for the Lombard effect involve a different approach from those of noise and distortion, being more related to methods for dealing with varying talker styles (quiet, loud, angry, etc). The interested reader is referred to Junqua and Angelade [20].

This section concentrates on addressing the issues of noise and channel effects. It is shown that methods of filtering the signal to remove the unwanted sounds are highly successful for constant, or slowly varying noise sources (e.g. line characteristics, or computer background noise), whereas for more complicated interfering noises (i.e. background speech, television or music) model adaptation techniques are required. These more advanced techniques have the overhead of higher computational requirements.

Communication channel distortion is a convolutional effect, and as such cannot be dealt with using the above mentioned techniques. The second half of this section describes a 'channel robust' speech parameterisation and its application to a speaker-verification system.

7.2 Noise robustness

Acoustic noise is usually considered to be additive. This means that in any transform domain which is linearly related to the original time-domain waveform, the speech and noise signals may be considered as separate sources having no cross-correlation:

$$F(\text{Speech} + \text{Noise}) = F(\text{Speech}) + F(\text{Noise})$$

where $F(.)$ is any linear transform.

Removal of the interfering noise, in theory, should not alter the characteristics of the speech signal in the linear

case. Unfortunately, the conventional MFCC representation of speech is not linearly related to the original waveform domain because of the log operation in the feature generation. Therefore noise robust recognisers operating in the MFCC or log spectral domains would typically invert the log transform before noise compensation is applied.

7.3 Stationary and non-stationary noise

An interfering noise source is termed 'stationary' if the statistics of the noise do not vary with time in the observation period. Examples of stationary noise include mains hum and fan noise from computer equipment. For slowly varying noise sources, it may be possible to update the noise parameters of the interfering signal, and hence track such a source. This type of noise may be termed 'quasi-stationary', and would represent, for example, the varying drone of the noise present in a car as its speed changes.

A typical example of 'non-stationary' noise would be a radio or television set playing in the background; it is not, in general, possible to update the noise parameters quickly enough to compensate for the changing nature of the noise.

7.4 Stationary-noise compensation techniques

One of the simplest techniques for dealing with stationary or quasi-stationary noise is that of **spectral subtraction** [21, 22]. This technique attempts to take account of the spectral shape of the interfering noise present in the signal, and operates in the linear power spectrum domain. An estimate of the noise mean in each channel is subtracted from the incoming signal; if the result of this process is less than a predefined spectral floor, the spectral floor value is substituted for the subtracted value. The clean speech estimate may be thus defined:

$$\hat{S}_k = \text{Max} ((S_k^2 + N_k^2) - \alpha N_k^2, \beta_k) \quad \dots\dots (7)$$

where S_k^2 and N_k^2 are the power spectra of the speech and noise components in frequency channel k respectively, β_k is the spectral floor in that channel and $\bar{N}_k^2$ is an estimate of the mean of the noise. Inclusion of the α factor in equation (7) allows the amount of noise energy subtracted from the signal to be varied.

To obtain an estimate of the noise mean, a method for discriminating speech from noise in a signal is required. One way to do this is to make use of a 'noise tracking' system [23] which makes decisions on a frame-by-frame basis, typically using an energy-based measure to determine the information content of the frame.

Determining appropriate values for the spectral floor threshold is very important to the success of the subtraction technique — the majority of non-speech frames will be replaced by the floor values in each channel, hence the spectral floor provides a desired 'environment independent' noise image. A set of spectral floor parameters may be determined by basing the threshold values on the dynamic range of the speech energy in each frequency channel, measured under ambient conditions. The floor values for each filterbank channel can then be set at a level below the majority of the energies associated with speech frames in each channel.

A drawback of this spectral floor is the distortion effect that the threshold has on the speech waveform; this is illustrated in Fig 18 where signal energies are compared in a particular channel before and after subtraction processing. The spectral floor for the channel is set at 32.5 dB, as shown by the dotted line. Also shown on the figure is the output produced by setting the spectral floor to zero, indicating the potential for severely distorting the speech if the floor is not carefully chosen. The effects of the signal distortion can significantly alter the cepstral representation of the speech data, it is therefore advantageous to include subtraction data during model training if cepstral parameters are to be used for recognition. In this case, a subsequent processing stage must be applied to the post-subtraction speech data in order to transform the parameters to the cepstral domain.

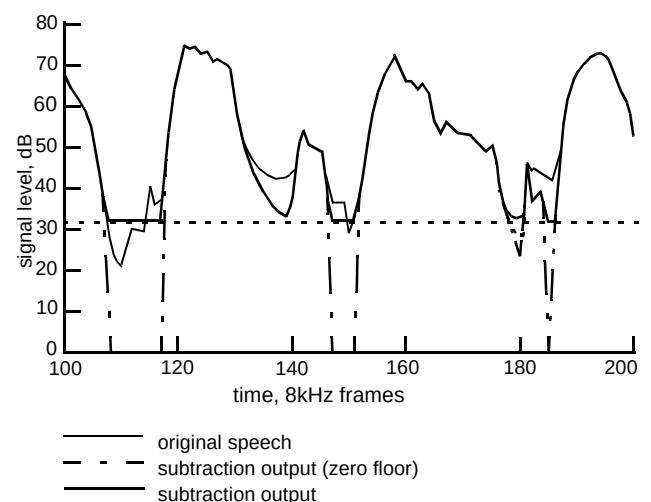


Fig 18 Time-domain distortion due to spectral subtraction.

Employing the spectral subtraction technique into the isolated-digit experiment of section 7.1 improves recognition performance in noisy conditions greatly — a recognition accuracy of over 75% is obtained at SNRs as low as 15 dB, a figure some 50% higher than the 'uncompensated' system performance. Figure 23 in section 7.6 plots the recognition accuracy as a function of signal-to-noise ratio.

If, instead of subtracting the noise estimate from the incoming data frames, it is used as a mask to screen out low energy detail from the signal, a second noise robust technique is obtained — ‘**noise masking**’ in this manner was first proposed by Klatt [24].

The masking is generally implemented by using the noise estimate in each frequency channel to replace the channel input if its value drops below that level. Similarly, the means of the speech models (trained on unmasked data) are adjusted so that any state output mean less than the associated noise estimate is replaced by the noise mean for that channel:

$$\text{INPUT: if } O_k < \mu_{nk} : \text{set } O_k = \mu_{nk} \quad \dots (8)$$

$$\text{HMMs: if } \mu_{sik} < \mu_{nk} : \text{set } \mu_{sik} = \mu_{nk} \quad \dots (9)$$

where O_k is the observation in a particular channel, μ_{nk} is the estimate of the noise mean in that channel and μ_{sik} is the mean of the channel pdf associated with state i of a particular speech HMM.

The ‘masked’ speech models are desensitised to the low-energy detail present in quiet training environments, but which would be swamped in noisy conditions. The recognition environment is thus stabilised in a predictable manner, leading to increased noise robustness.

One inherent advantage of the masking process over that of spectral subtraction is that it is applied in the log-filterbank domain, thus requiring less computational overhead in data transformation if recognition is being carried out in the cepstral domain. Figure 19 details the processing stages required.

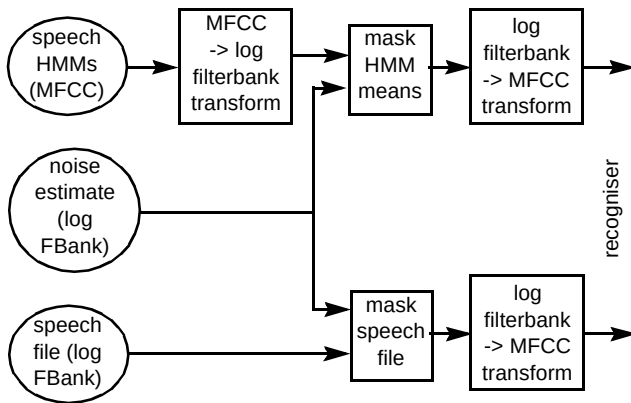


Fig 19 Transform stages in the noise masking system.

Implementing the masking technique on the benchmark isolated digit experiment results in similar performance gains to those obtained with spectral subtraction. The results, shown in Fig 23 in section 7.6, show an increase in useful operational range of the recogniser compared to the

earlier method, providing word accuracies of 80% at 15 dB SNR, and over 70% as low as 10 dB.

The performance of both systems declines quickly as the signal-to-noise ratio falls below 10 dB. This occurs when useful information in the speech signal starts to be removed due to either subtraction or masking with increasingly large noise estimates. Subtraction data will become steadily more distorted, and result in a poorer match with the speech models.

The effect on the noise-masking technique is twofold. Firstly, as the noise estimate increases, more and more speech model means will be masked, and hence their discriminational ability will be reduced. Secondly, masking of the incoming utterances has the effect of reducing the variance of the non-speech regions of the data, and hence reducing the variance of the ‘noise’. A consequence of this is that the recognition process will increasingly favour word models over noise models due to the tighter variances associated with speech models (variances are not altered by the masking technique).

7.5 Severe and non-stationary noise compensation

The loss of low-energy detail in noisy conditions can be clearly illustrated by comparing the log filterbank HMM ‘profiles’ of a speech model trained in both clean and noisy conditions. Figures 20 and 21 present such profiles for the isolated-digit model ‘eight’: the plots show the log filterbank energy (z-axis) for each frequency coefficient (y-axis) as a function of model state (x-axis). Hence Figs 20 and 21 represent the change in characteristic spectral shape of the model over the duration of the model. Fig 22 illustrates the adapted model.

Figure 20 represents the ‘clean’ version of the digit, and can be seen to have clearly defined contours in the low energy regions. In comparison, the ‘matched’ model of Fig 21, trained in severe noise conditions (i.e. below 10 dB SNR), has lost large amounts of the low energy detail which can characterise a particular word.

Achieving the ‘matched’ system levels of performance in these conditions requires some method for simulating the effects of training the recogniser in these environments. Various methods for achieving this have been reported in the literature [6, 25]. The latter presents a technique known as ‘**Wiener Adaptation**’, an implementation of Wiener filtering [26] in the context of an HMM.

The conventional Wiener filter equation operates in the spectral domain, where the filtered speech signal is given by:

$$\hat{S}(f) = Y(f)W(f) = Y(f) \frac{\mu_s(f)}{\mu_s(f) + \mu_N(f)} \quad \dots (10)$$

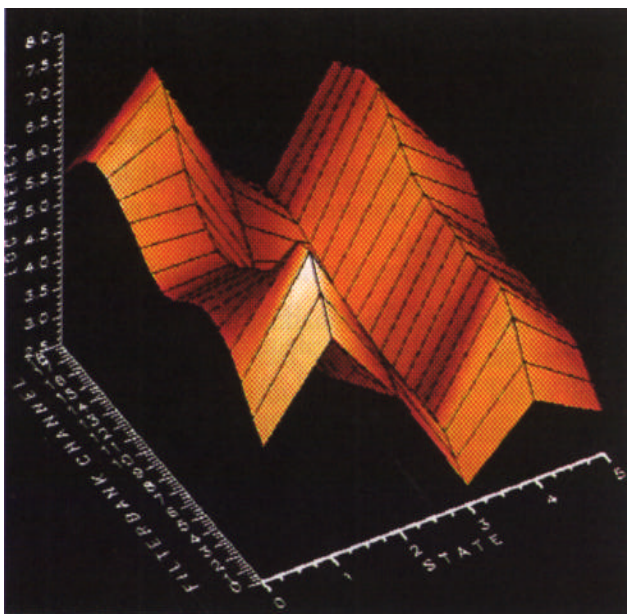


Fig 20 Log filterbank profile for HMM 'eight' trained in clean conditions.

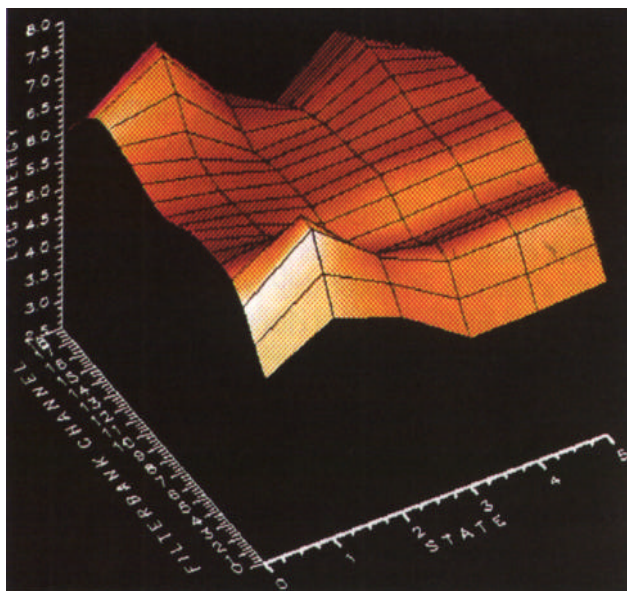


Fig 21 Log filterbank profile for HMM 'eight' trained in 'matched' conditions.

where $\mu_s(f)$ and $\mu_n(f)$ represent the mean of the clean speech and interfering noise spectra respectively. The noisy speech signal is denoted by $Y(f)$.

It is shown in Milner and Vaseghi [25] that Wiener filtering of the noisy speech is equivalent to the adaptation of the mean cepstral vectors of the states of an HMM. The adaptation technique overcomes the two key drawbacks of the Wiener filter, namely that:

- the filter assumes speech is stationary (this is true in the HMM context, as each state represents a quasi-stationary event),

- a clean speech estimate is available (the HMM means provide such 'clean' speech estimates).

Modifying the clean speech models using the technique effectively adapts the speech models to an approximation of the current environment, thus simulating 'matched' conditions as required. Using the adaptation method, the 'clean' model 'eight' of Fig 20 was adapted to the adverse conditions used for training the 'matched' model of Fig 21 (the same method for obtaining the noise estimate as in the earlier techniques is applicable). The 'adapted' model is illustrated in Fig 22 for comparison with the 'matched' HMM.

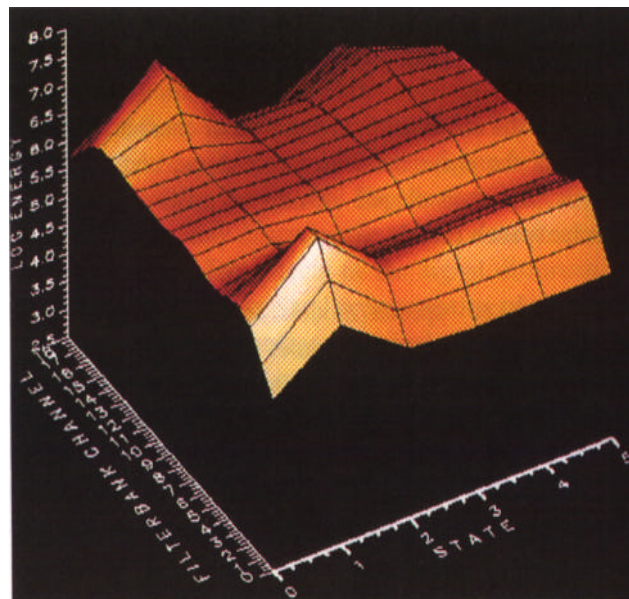


Fig 22 Log filterbank profile for adapted HMM 'eight'.

Adding Wiener adaptation into the recogniser for the isolated-digit experiment delivers the best overall performance, achieving a word accuracy of over 80% at 10 dB SNR, a figure within 8% of the matched performance and improvement of over 63% from the original uncompensated system. The results are again shown on Fig 23.

7.6 Noise-robust results and conclusions

A comparison of the various methods for dealing with adverse conditions has been made. The relative performance of each method is compared using a baseline HMM-based isolated-digit recogniser.

Noisy conditions were simulated by adding noise data recorded in a car travelling at 60 mph to the digit utterances at various levels, this particular noise type has a quasi-stationary spectrum.

The results, shown in Fig 23, clearly indicate the benefit of adding noise robustness to a recognition system if it is to operate successfully in a 'real-world' situation. The trade-off between recognition performance and computational

complexity of the robust techniques is also captured by the graph.

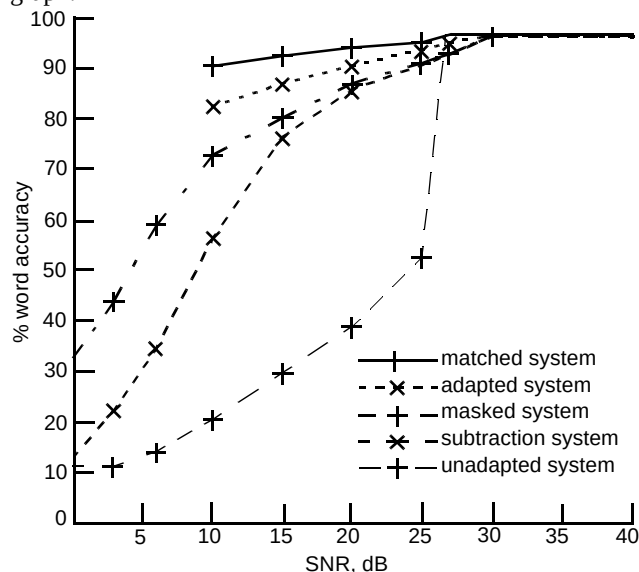


Fig 23 Results of the isolated digit experiments for various noise robust techniques.

The Wiener adaptation system consistently achieves the highest recognition accuracy over all the conditions tested here, and has been shown to compete well with the more complicated model adaptation techniques [6, 27] used for highly non-stationary noise.

7.7 Channel robustness

Differences in training and testing conditions are known to cause degradation in the performance of speaker recognition systems [28]. A major source of signal variability occurs as a consequence of differing frequency responses between telephone channels. There are many factors which affect the characteristics of a communication channel such as line length, handset model and telephone exchange type. Although these effects do produce higher order interactions the spectral degradation found in speech collected over a PSTN can be modelled as a combination of invertible linear filters representing the various telephony components.

The following section investigates the effect of non-ideal channel frequency responses on the accuracy of telephony-based speaker recognition systems. It is shown that a technique known as short-term cepstral mean normalisation, an alternative to long-term cepstral mean normalisation [29], increases performance while reducing the amount of enrolment data required.

7.8 Speaker recognition — an overview

Automatic speaker recognition can be defined as the process of recognising individuals from characteristic

acoustic features of their voice. The subject area is divided into two categories:

- speaker verification is the process of authentication of a person's identity claim by comparing a sample of their speech to that of the claimed identity,
- speaker identification is the process of attributing the speech produced by an unknown speaker to one of a population of known speakers for whom speech samples are available.

7.9 The cross-channel effect

It is of particular interest to consider the effect of a linear filter on MFCCs, since they are popular for speaker recognition. If the channel is treated as a combination of linear filters, then the observed time-domain signal $y(t)$ is given by:

$$y(t) = x(t) * h(t) \quad \text{..... (11)}$$

where:

$x(t)$ is the speech signal prior to filtering,

$h(t)$ is the overall impulse response of the system,

In the Mel-filterbank domain the convolutional channel distortion becomes multiplicative:

$$Y(f) = X(f)H(f) \quad \text{..... (12)}$$

where f is the filterbank channel.

In the log filterbank domain the channel distortion becomes additive and, because of the linearity of the DCT, the effect of the channel in the cepstral domain is also additive:

$$c_y(n, m) = c_x(n, m) + c_h(n) \quad \text{..... (13)}$$

where, $c_y(n, m)$ is the n th MFCC coefficient of the m th time frame and $c_x(n, m)$ is the n th MFCC coefficient of the m th time frame before passing through channel with distortion vector $c_h(n)$.

7.10 Channel compensation techniques

Speaker recognition requires that an individual's utterance is compared against one or more speaker models. If the utterances used to create the models are not collected over the same telephone channel, the values obtained from distance calculations will have an element of error dependent upon the channel variation between models.

7.10.1 Channel averaging

A reasonable strategy for dealing with the channel effect is to train the speakers' models with speech samples from several different channels and thereby create a set of 'channel averaged' speaker representations. The disadvantage of this approach is that it is not often practical to ask an individual to perform several enrolment sessions. In addition, performance can be reduced when channel effects are averaged together since channel variations can obscure utterance variations.

7.10.2 Differential features

Another technique for removing the channel effect is the use of differential feature vectors. Although these do remove the channel effect they have also been shown to remove valuable speaker information. When used in isolation, they do not perform as well as static features [29].

7.10.3 Cepstral mean normalisation

A well-documented method for dealing with the channel effect is long-term average cepstral mean normalisation [30]. This technique attempts to find an estimate of the channel distortion cepstral vector by calculating a long-term average over the entire length of a sentence-long utterance and subtracting this from each vector of the signal.

The accuracy of the estimate depends on the amount of data available. The global estimate, L , is given by averaging over the vectors in the sentence-long utterance:

$$L = \frac{\sum_{m=1}^N c_y(m)}{N} \quad \text{..... (14)}$$

where $c_y(m)$ is the m th MFCC vector, and N is the number of vectors in the sentence.

Each MFCC vector can be thought of as being made up of a time-invariant component, i.e. the overall channel distortion vector, and a time-varying component — thus:

$$c_y(m) = c_x(m) + c_h \quad \text{.....(15)}$$

where $c_x(m)$ is the time varying component of the m th MFCC vector, and c_h is the overall time-invariant channel-distortion vector.

Substituting equation (15) into equation (14), the global estimate, L , is given by:

$$L = \frac{\left(\sum_{m=1}^N (c_x(m) + c_h) \right)}{N} \quad \text{..... (16)}$$

that is:

$$L = \frac{\sum_{m=1}^N c_x(m)}{N} + c_h \quad \text{..... (17)}$$

as N increases, the sentence mean approaches zero:

$$\frac{\sum_{m=1}^N c_x(m)}{N} \rightarrow 0 \quad \text{..... (18)}$$

and:

$$L \rightarrow c_h \quad \text{..... (19)}$$

In cases where equation (18) does not sum to zero (because the sentence is not long enough), the channel estimate will be biased by the sentence mean. This bias would be harmless if for a given speaker it was identical over all versions of the sentence. Unfortunately, this is unlikely to be the case due to intra-speaker variability.

7.11 Short-term cepstral mean normalisation

As an alternative to normalising over the entire length of the utterance, short-term cepstral mean normalisation produces channel-robust features without directly estimating the channel distortion.

Whereas the long-term cepstral mean-normalisation technique attempts to estimate the channel-correction vector, short-term normalisation normalises over isolated words, between the speech endpoints, and ignores frames where only background noise is present — the low-energy content gives a very poor spectral estimation.

The short-term normalisation process removes not only the effect of the channel but also some speech and speaker information. It might, therefore, be thought that this would reduce the recognition accuracy by removing data which could be used to distinguish between speakers. Surprisingly, this is not the case. Removal of the short term mean simply sets the distortion to a common level instead of removing it altogether. This is due to the fact that although equation (18) does not sum to zero over an isolated word, the variability between versions is much less than that for a long sentence. Thus the estimate given by equation (17) will be consistent across different instances of the utterance.

The short-term normalisation technique is illustrated in Fig 24. The left-hand column shows a trace of MFCC coefficient value over the time duration of an isolated utterance. In each case two traces are shown; these correspond to the same recorded isolated utterance

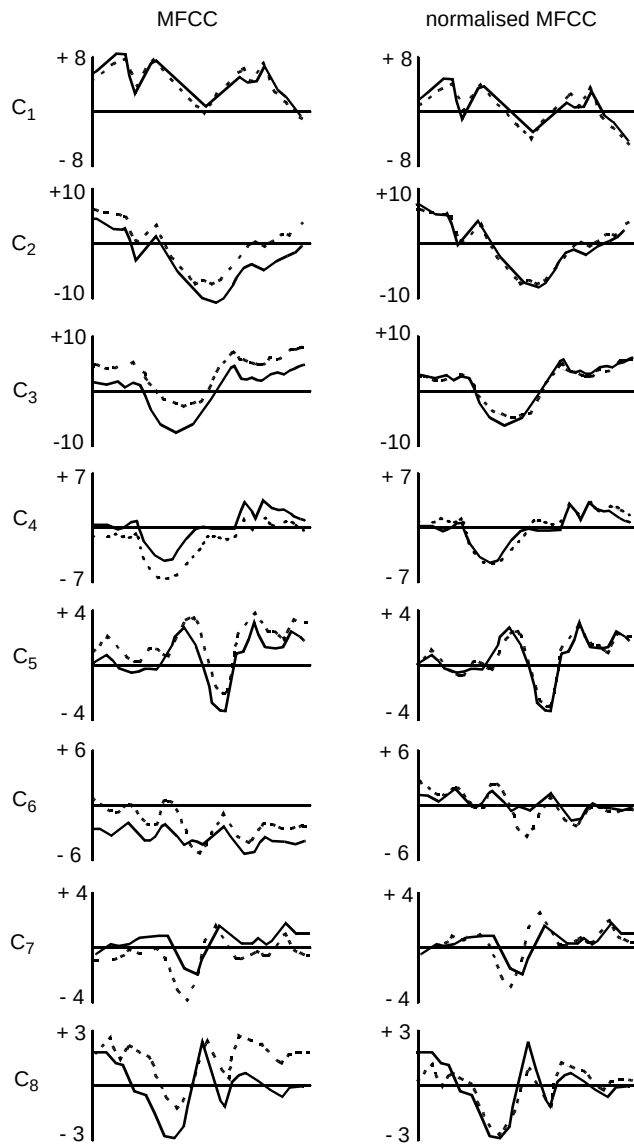


Fig 24 Comparison between MFCCs and normalised MFCCs for speech transmitted through two channels.

transmitted via two different transmission channels. The channel effect produces a constant offset between the two traces, corresponding to the difference in transfer characteristic in the frequency bands between the two channels.

Referring to the right-hand column, the plots of the normalised MFCCs are shown. It can be seen that the normalisation has brought the traces into closer alignment.

7.12 Experiments

This section describes a set of speaker-identification and speaker-verification experiments designed to compare the performance of MFCCs with short-term cepstral mean normalised MFCCs.

7.12.1 Speech database

The experiments use a subset the BT Brent telephony database. The subset consists of 56 versions of the full isolated-digit set ('one' to 'nine', 'zero', 'nought' and 'oh') from 100 talkers. The first 12 versions are reserved for training and the remaining 44 versions (1 recorded per week from various locations) are for testing. The training sets are divided into the categories shown in Table 7.

Table 7 Training set categories.

Training sets	Description
1	Three utterances in a single call
2	Three utterances in three calls in a single week
3	Three utterances in three calls in three weeks
4	Eight utterances in two calls in two weeks (versions 1—4 from single call, versions 5—8 from separate calls)
5	Eight utterances in three calls in three weeks

The training set categories simulate different enrolment scenarios. Set 1, for example, represents a single session, which would be expected to give poorer performance than several sessions over several days.

7.12.2 Recognition system

A DTW classifier was used in preference to HMM or VQ codebook since this has been shown to give the best performance for the task of Speaker Recognition [31].

The individual digits were endpointer using an energy based endpointer.

Reference templates were constructed as follows. From the enrolment data the median-length utterance was designated as the initial template to which the others are time aligned using DTW. After alignment the utterances are averaged together into a single template.

Recognition was performed using a 'city-block' distance measure between the template and the test token. A single test token consists of the full isolated-digit set. A distance score was obtained for each of the twelve digits and an overall score calculated by averaging the individual scores together.

Each recognition experiment consists of 4400 test tokens (44 from each speaker). For speaker verification, speech from all talkers is matched against all individuals'

models, and the resulting scores divided into true-talker and imposter scores; true-talker scores being when the test token is from the same speaker as was used to generate the models (simulating true-talker identity claims), with the remainder imposter scores (simulating imposter identity claims). The score values are used to calculate individual equal-error rates for all speakers, which are then averaged across the entire speaker set to give the overall performance figure. Speaker identification is simulated by presenting each test token to all models and choosing the best match.

7.12.3 Results and discussion

A comparison of speaker-recognition performance between MFCCs and short-term cepstral mean normalised MFCCs has been made, using isolated-digit input.

For each of the five training sets, verification and identification performance is given in Figs 25 and 26.

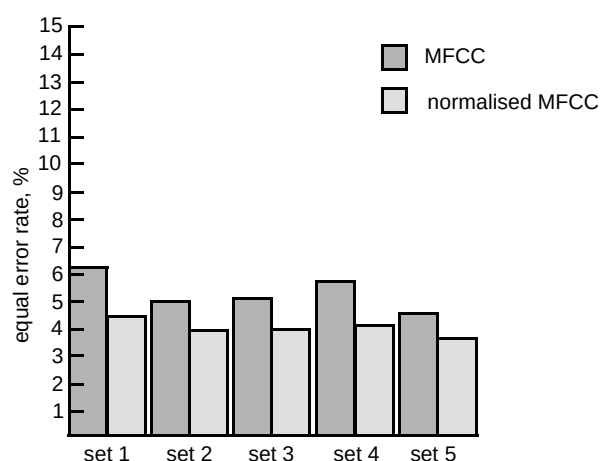


Fig 25 Speaker verification performance for the five training sets.

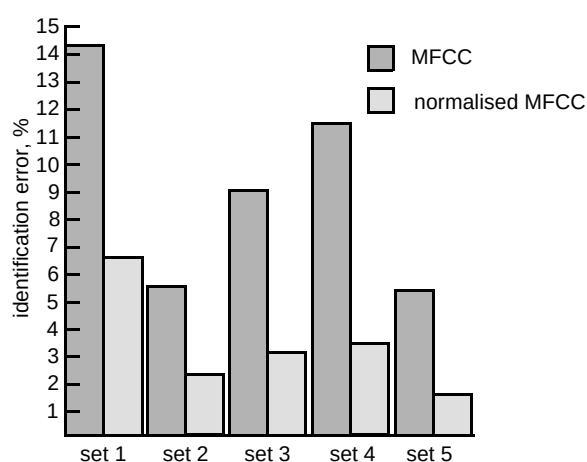


Fig 26 Speaker identification performance for the five training sets.

The results show that the use of short-term normalised cepstra give an overall performance improvement. Although the ranking between the performance of the various training sets is the same (set 1 gives poorest performance followed by sets 4, 3, 2, and 5), the normalised cepstra decrease the error rates and reduce the relative differences between sets.

For speaker verification it can be seen that the use of normalised cepstra obviates the need for multiple enrolment sessions. Irrespective of the training set used, the difference in performance is not statistically significant. It is also interesting to note that in the use of unnormalised cepstra the performance obtained when training with set 4 does not outperform that when using set 3. Set 4 was collected over two rather than three calls, but does have an additional five repetitions of the vocabulary and so may be expected to perform better.

For speaker identification it can be seen that the normalised cepstra substantially improve performance. This indicates that speaker identification is very susceptible to differences between channels. This is not surprising since the identification process requires that test tokens are compared against all speaker models. If the models and/or test tokens are biased by an unknown distortion vector, it is not unreasonable to expect additional errors.

8. Summary

This paper has shown new and powerful techniques which enable speech and speaker recognition technologies to operate successfully in a variety of environments. The key constituents of a robust recogniser have been considered and several technological advances have been demonstrated:

- a new feature extraction technique, namely the cepstral-time matrix, gives higher recognition accuracy over the conventional cepstral vector augmented with higher-order time derivatives,
- a dynamic recogniser which provides an efficient solution to dealing with large vocabularies or long utterances,
- improved recognition accuracy via phoneme-based, context-dependent modelling techniques,
- user selectable vocabulary using subword HMMs,
- increased OOV rejection via the use of sink models,
- noise and channel robustness via a combination of model adaptation and feature normalisation techniques.

Although these techniques provide the basis for constructing sophisticated recognition systems there is still scope for improvement in many of the areas. Nevertheless this work is among the most advanced in its field and enables BT to provide state-of-the-art recognition systems.

Acknowledgements

The authors wish to acknowledge the work of former team members who have contributed to BT's corporate speech recognition research programme, Dave Chaplin, Stephen Cox, Steve Johnson, Denis Johnston, Bob Linggard, William Millar, John Oglesby, Frank Scahill, Gavin Smyth, John Talintyre, Joseph Tang and Geoff Walker.

References

- 1 Westall F A et al: 'Speech technology in telecommunications', *BT Technol J*, **14**, No 1, pp 9—27 (January 1996).
- 2 Power K J: 'The listening telephone — automating speech recognition over the PSTN', *BT Technol J*, **14**, No 1, pp 112—126 (January 1996).
- 3 Parsons T: 'Voice and speech processing', McGraw-Hill (1987).
- 4 Hanson B A and Applebaum T H: 'Robust speaker-independent word recognition using static, dynamic and acceleration features: experiments with Lombard and noisy speech', *Proc ICASSP*, pp 857—860 (1990).
- 5 Milner B P: 'Speech recognition in adverse environments', PhD Thesis (1994).
- 6 Gales M J F and Young S J: 'An improved approach to the hidden Markov model decomposition', *Proc ICASSP*, pp 729—732 (1992).
- 7 Woodland P C, Odell J J, Valtchev V and Young S J: 'Large vocabulary continuous speech recognition using HTK', *Proc ICASSP*, pp 125—128 (1994).
- 8 Jelinek F: 'Continuous speech recognition by statistical methods', *Proc IEEE*, **64**, No 10, pp 532—556.
- 9 Katz S M: 'Estimation of probabilities from sparse data for the language model component of a speech recogniser', *IEEE Trans ASSP*, **35**, No 3, pp 400—401 (March 1987).
- 10 Kenn P, Hollan R, Gupta V N, Lennig M, Mermelstein P and O'Shaughnessy D: 'A* — admissible heuristics for rapid lexical access', *IEEE Trans Speech and Audio Processing*, **1**, pp 49—58 (January 1993).
- 11 Ney H: 'Search strategies for large-vocabulary continuous-speech recognition', in Rutio and Soler (Eds): 'Speech Recognition and Coding (New Advances and Trends)', Springer (1993).
- 12 Schwartz R and Austin S: 'A comparison of several approximate algorithms for finding multiple (N_BEST) Sentence Hypotheses', *Proc ICASSP*, pp 701—704 (1991).
- 13 Hwang M, Huang X D and Alleva F: 'Predicting unseen triphones with senones', *Proc ICASSP*, pp 311—314 (1993).
- 14 Huang X D and Jack M A: 'Semi-continuous hidden Markov models for speech recognition', *Computing Speech and Language*, **3**, pp 239—251 (1989).
- 15 Lee K F: 'Large vocabulary speaker-independent continuous speech recognition', The SPHINX system, PhD Thesis, CMU-CS-88-148 (1988).
- 16 Young S J: 'Benchmark DARPA RM results with the HTK portable HMM toolkit', *Proc DARPA Workshop* (September 1992).
- 17 Paul D B: 'The Lincoln continuous speech recognition system: recent developments and results', *DARPA Speech and Natural Language Workshop*, pp 160—166 (February 1989).
- 18 Young S J: 'HTK version 1.3: reference manual', Cambridge University Engineering Dept, Speech Group (May 1992).
- 19 Svendsen T, Soong F and Purnhagen H: 'Optimizing baseforms for HMM-based speech recognition', *Proc Eurospeech*, pp 783—786 (1995).
- 20 Junqua J C and Angelade Y: 'Acoustic and perceptual studies of Lombard speech', *Proc ICASSP*, pp 841—844 (1990).
- 21 Van Compernelle D: 'DSP techniques for speech enhancement', *Proc Speech Processing in Adverse Conditions* (1992).
- 22 Munday E: Chapter 6 in Wheddon C (Ed): 'Speech and language processing', Chapman & Hall (1990).
- 23 Bridle J S, Ponting K, Brown M and Borrett A: 'A noise compensating spectrum distance measure applied to automatic speech recognition', *Proc IOA*, pp 307—310 (1984).
- 24 Klatt D H: 'A digital filterbank for spectral matching', *Proc ICASSP*, pp 573—578 (1976).
- 25 Milner B P and Vaseghi S V: 'Comparison of some noise-compensation methods for speech recognition in adverse environments', *Proc IEE*, **141**, pp 280—288 (1994).
- 26 Wiener N: 'Extrapolation, interpolation and smoothing of stationary time series with engineering applications', MIT press (1949).
- 27 Varga A and Moore R K: 'Hidden Markov model decomposition of speech and noise', *Proc ICASSP*, pp 845—848 (1990).
- 28 Gish H: 'Investigation of text-independent speaker identification over telephone channels', *Proc ICASSP*, pp 379—382 (1985).
- 29 Ariyaeeinia A M and Sivakumaran: 'Transitional features', *IEEE International Carnahan Conference on Security Technology*, pp 79—84 (1995).
- 30 Furui S: 'Cepstral analysis technique for automatic speaker verification', *IEEE Trans Speech and Signal Processing*, **29**, No 2, pp 254—272 (April 1981).
- 31 Yu K, Mason J and Oglesby J: 'Speaker recognition models', *Proc Eurospeech*, pp 629—632 (1995).



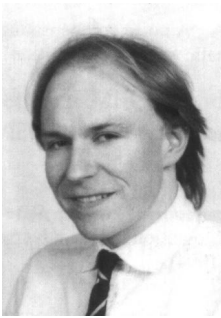
Mark Pawlewski graduated with a BSc in natural philosophy from Glasgow University in 1984. He went on to obtain a postgraduate diploma in offshore engineering from Robert Gordon's Institute of Technology, Aberdeen, in 1985. After three years in the oil industry, working in subsea navigation, he returned to Strathclyde University and completed an MSc in information technology in 1989. He subsequently joined the speech technology unit at BT where his work has mainly focused on speaker recognition research. He is currently a project leader of the speech and speaker recognition research project.

James Bridges graduated from Reading University in June 1992 with a BSc in Cybernetic and Control Engineering. He went on to join the speech technology unit at BT working on the corporate speech recognition research project. The majority of his work has concentrated on the development of algorithms to make recognisers more robust to real world applications. From October 1995 he became an employee of Rover. He is an associate member of the IEE.



Simon Downey graduated with a BEng in electrical and electronic engineering from Bristol University in 1990. He joined the speech research unit at DRA Malvern in 1991, working on large vocabulary speech recognition systems and a segment-based speech vocoder.

Since joining the speech technology unit of BT Laboratories in 1993 his work has been focused mainly on noise robustness and speaker recognition.



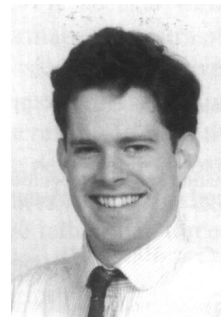
Simon Hovell graduated with a BEng in Electrical and Electronic Engineering from Edinburgh University in 1990. He went on to obtain a PhD in Signal Processing (also from Edinburgh University) in 1994, which involved research into higher order spectra and nonlinear techniques for characterisation and modelling. He then joined the speech Technology Unit at BT Laboratories where he is currently involved in research into large-vocabulary and natural-language speech recognition. He is a member of the IEEE.



Ben Milner received a PhD in signal processing from the University of East Anglia in 1995. Prior to this he obtained a BEng in electronic engineering in 1991 and spent some time in the oil industry, working on side scan sonar and seabed profiling. Since 1994 he has been working at BT Laboratories in the speech recognition group, specialising in noise and channel robustness and front-end processing.

He is also an associate member of the IEE and the visiting signal processing lecturer at the University of East Anglia.

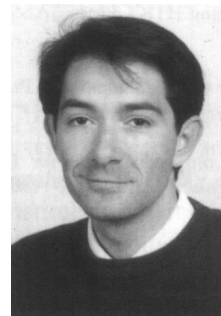
David Ollason graduated from Edinburgh's Heriot-Watt University in June 1990 with an MEng in electrical and electronic engineering. He joined the speech technology unit at BT in September 1990 working on the corporate speech recognition research project. His work has mainly focused on the development of sub-word unit acoustic modelling but latterly has concentrated on parser design for continuous speech recognition decoders. As of June 1995 he became an employee of Entropic Cambridge Research Laboratories.



Kevin Power graduated from University College, Cork, Ireland in 1990 with a degree in electrical and electronic engineering. He worked for Hyperion Energy Systems in Cork developing the waveform definition component of a radar control system before joining the speech technology unit at BT Laboratories in January 1991.

His initial work involved downstream speech recognition algorithms. In 1993 he joined the speech and speaker recognition research team where he developed techniques for 'out of vocabulary' rejection.

Since then he has been investigating improved acoustic models for recognition. He is currently a project leader of the speech and speaker recognition research project.



Simon Ringland received the degree of BA in electrical sciences from Cambridge University in 1986. He then joined BT Laboratories and began work on speech recognition algorithm research. In 1991 he led a team developing speech recognition for an automated directory enquiries trial system. He currently heads up BT's speech and speaker research team which develops algorithms aimed primarily at telephony-based interactive voice response systems, such as those used in the BT CallMinder™ service.